

AI Chatbot & RAG Document Assistant

AI-Powered Conversational and Document Retrieval System

BY

Vaishnav R

TECHNOLOGIES

Groq

LangChain

ChromaDB

Streamlit

HuggingFace

Project Overview

Conversational AI

Answer questions naturally through multi-turn conversations

PDF Document Retrieval

Upload PDFs and extract precise, contextual answers

RAG Architecture

Retrieval-Augmented Generation for accurate responses

Context-Aware Responses

Understands document context and conversation history

Business Problem

Unmanageable Documents

Large documents are impossible to search manually at scale

Time-Consuming Search

Finding relevant info wastes valuable working hours

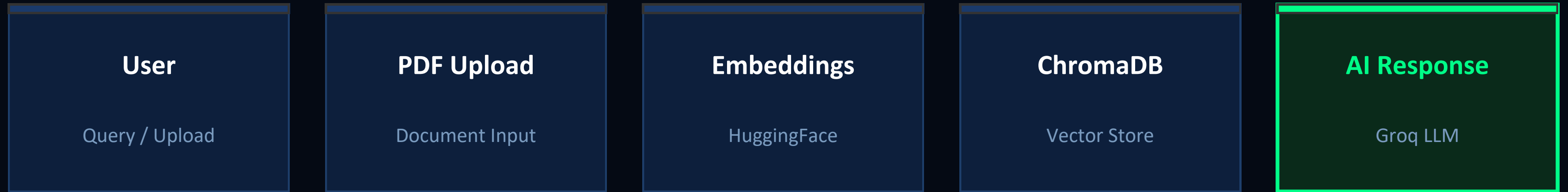
Repetitive Queries

Same questions answered repeatedly, reducing productivity

Lack of Context

Traditional search misses semantic meaning and relationships

Proposed Solution



Key Benefits

- Conversational AI assistant
- Semantic document retrieval
- Context-aware responses
- Multi-document support
- Faster information access

Task 1 — Basic AI Chatbot

Multi-Turn Conversations

Maintains full conversation context across messages

Conversation Memory

Remembers prior context for coherent dialogue

Groq LLM Integration

Blazing-fast inference with Groq API

Real-Time Responses

Instant streaming answers to user queries

Adjustable AI Behavior

Customizable system prompts and temperature

```
$ python chatbot.py
```

```
AI Chatbot Ready! Type 'exit' to quit.
```

```
You: What is RAG?
```

```
AI: RAG stands for Retrieval-  
Augmented Generation. It combines  
retrieval with generation to
```

```
You: _
```

Task 2 — RAG AI Document Assistant

PDF Upload Interface

Semantic Search

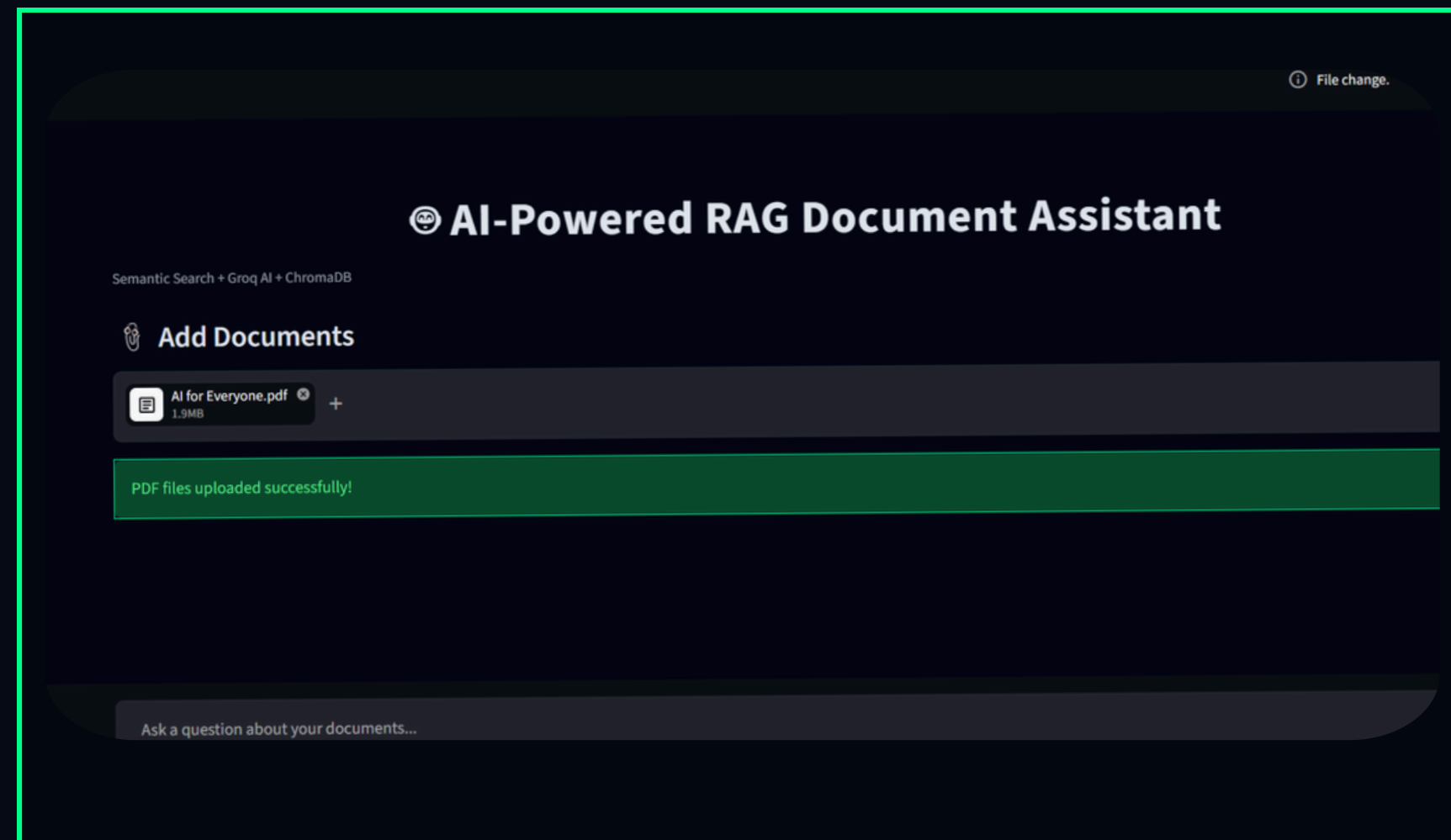
ChromaDB Vector DB

Source Chunk Tracking

Multi-Document Retrieval

Streamlit Web Interface

Conversational Memory



Technologies Used



Groq

AI RESPONSE GENERATION

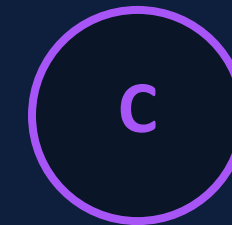
Ultra-fast LLM inference API for real-time answers



LangChain

RAG WORKFLOW

Orchestrates retrieval chains and LLM pipelines



ChromaDB

VECTOR DATABASE

Stores and queries document embeddings efficiently



HuggingFace

EMBEDDING GENERATION

Semantic text embeddings for document indexing



Streamlit

WEB INTERFACE

Rapid web app framework for the AI frontend



Python

BACKEND DEVELOPMENT

Core language powering all system components

Challenges & Solutions

⚠ CHALLENGE

● Slow document retrieval

● Context handling across turns

● Dynamic PDF uploads

● Accurate semantic retrieval



✓ SOLUTION

● Optimized chunking strategy

● Conversational memory buffer

● Real-time indexing pipeline

● Embedding-based semantic search

My Contribution

01

Chatbot Backend

Developed core chatbot backend logic and conversation handling

02

Groq API Integration

Integrated Groq LLM API for high-speed inference

03

RAG Pipeline

Implemented full Retrieval-Augmented Generation pipeline

04

Streamlit Frontend

Built responsive and interactive web UI with Streamlit

05

PDF Upload System

Added real-time PDF upload and indexing capability

06

Source Tracking & Memory

Enabled chunk-level source tracking and memory

07

Futuristic UI Design

Designed the dark AI-themed user interface

Future Improvements

1

Hybrid Retrieval

Combine semantic + lexical search for better accuracy

2

Better Caching

Smarter caching layer to reduce response latency

3

Streaming Responses

Token-level streaming for real-time AI output

4

Voice Input Support

Speech-to-text interface for voice queries

5

Cloud Deployment

Deploy on AWS / GCP with auto-scaling support

6

Advanced Reranking

Cross-encoder reranking for more precise results

Thank You